University of Exeter

ECM2433: The C Family
C++: operator overloading

Operator overloading: background

University of Exeter

Good idea to have a comment here to explain why year is “int”, but month and day are “unsigned int”.

Consider this date class:

```
class MyDate
{
   int             year;
   unsigned int    month;
   unsigned int    day;
 public:
   MyDate(unsigned int, unsigned int, int);
}

MyDate::MyDate (unsigned int day, unsigned int month, int year)
{
   this->day   = day;
   this->month = month;
   this->year  = year;
}
```

```
int main ()
{
   // 1st March 2021
   MyDate dt(1,3,2021);
   return 0;
}
```

This constructor should do more than this; what else should it do?

Operator overloading: background

Given these declarations:

```
MyDate dt1(29,6,2015);
MyDate dt2(1,3,2021); int
       diff;
MyDate dt3;
```

It would be nice to be able to do this:

```
diff = dt2 - dt1;
dt3  = dt2 + 43;
```

Rather than something like this:

```
diff = dt2.difference(dt1); dt3
     = dt2.addDays(43);
```

Operator overloading: background

University of Exeter

Given these declarations:

```
MyDate dt1(29,6,2015);
MyDate dt2(1,3,2021);
int    diff;
MyDate dt3;
```

It would be nice to be able to do this:

```
diff = dt2 - dt1;
dt3  = dt2 + 43;
```

Operator overloading
more than one “meaning” of an operator

Rather than something like this:

```
diff = dt2.difference(dt1);
dt3  = dt2.addDays(43);
```

Compare with: function overloading
more than one function with the same name

Operator overloading

The aim of operator overloading it to make life easier for users of a class, not for the developer of the class.

Easier means:

- Easier to understand the code
- A more natural style of programming, closer to the domain
- Fewer code defects

Operator overloading

The following operators cannot be overloaded:

```
.     sizeof     ?     ::     .*
```

All the others can, e.g.:

```
+     %     ()     []     <=     <<     ++     &     new     delete
-     /     !=     ->     &&     |=     ^     +=     new[]     delete[]
```

see e.g. Wikipedia for a full list:
https://en.wikipedia.org/wiki/Operators_in_C_and_C%2B%2B

Operator overloading:

University of Exeter

Syntax:

```
return_type operator<symbol>(parameter_list);
```

```
class MyInteger
{
      int    i;
   public:
      MyInteger (int i) { this->i = i; }

      int    operator+(MyInteger);
};

// This function represents <MyInteger> + <MyInteger>
int MyInteger::operator+ (MyInteger m)
{
   return this->i + m.i;
}
```

```
int main ()
{
   MyInteger a(4);
   MyInteger b(3);
   MyInteger c;

   c = a + b;

   return 0;
}
```

Operator overloading: +

```
class MyInteger
{
      int    i;
   public:
      MyInteger (int i) { this->i = i; }
      int    operator+(MyInteger);
      int    operator+(int);
};

// This function represents <MyInteger> + <MyInteger>
int MyInteger::operator+ (MyInteger m)
{
   return this->i + m.i;
}

// This function represents <MyInteger> + <int>
int MyInteger::operator+ (int a)
{
   return this->i + a;
}
```

```
int main ()
{
   MyInteger a(4);
   MyInteger b(3);
   MyInteger c;

   c = a + b;
   c = c + 6;


   return 0;
}
```

Operator overloading: +

```
class MyInteger
{
      int    i;
   public:
      MyInteger (int i) { this->i = i; }
      int    operator+(MyInteger);
      int    operator+(int);
};

// This function represents <MyInteger> + <MyInteger>
int MyInteger::operator+ (MyInteger m)
{
   return this->i + m.i;
}

// This function represents <MyInteger> + <int>
int MyInteger::operator+ (int a)
{
   return this->i + a;
}
```

```
int main ()
{
   MyInteger a(4);
   MyInteger b(3);
   MyInteger c;

   c = a + b;
   c = c + 6;
   c = 8 + c;

   return 0;
}
```

But what about <int> + <MyInteger>?

Operator overloading: +

```
class MyInteger
{
        int    i;
    public:
        MyInteger (int i) { this->i = i; }
        int    operator+(MyInteger);
        int    operator+(int);
};
```

But what about <int> + <MyInteger>?

```
int operator+ (int a, MyInteger m)
{
    return a + m.i;
}
```

✗ `i` is private

```
int main ()
{
    MyInteger a(4);
    MyInteger b(3);
    MyInteger c;

    c = 8 + c;



    return 0;
}
```

Solution: friend functions

In relation to a class, a **friend** function is

- A non-member function
  - i.e. a function that is not a member of the class
- Which has access to the private and protected elements of the class
  - i.e. the class for which it is a friend

Friend functions

In relation to a class, a **friend** function is

- A non-member function
  - i.e. a function that is not a member of the class
- Which has access to the private and protected
  elements of the class
  - i.e. the class for which it is a friend

Friendship is **<u>not</u>**

- Inherited
  - your friend's children are not your friends

Friend function: example

```
class MyClass
{
        int    a,b;
    public:
        MyClass(int, int);
        friend int sum(MyClass);
};

MyClass::MyClass(int a, int b)
{
    this->a = a;
    this->b = b;
}

int sum(MyClass x)
{
    return x.a + x.b;
}
```

sum is not a member of MyClass, or any other class

but can access the private members of MyClass

```
int main ()
{
    MyClass n(3,4);
    cout << sum(n) << endl;
    return 0;
}
```

Operator overloading: friend function

```
class MyInteger
{
      int    i;
   public:
      MyInteger (int i) { this->i = i; }

      int          operator+(MyInteger);
      int          operator+(int);
      friend int operator+(int, MyInteger);
};

// This function represents <MyInteger> + <MyInteger>
int MyInteger::operator+ (MyInteger m) { ... }

// This function represents <myInteger> + <int>
int MyInteger::operator+ (int a){ ... }

// This function represents <int> + <MyInteger>
int operator+ (int a, MyInteger m)
{
   return a + m.i;
}
```

```
int main ()
{
   MyInteger a(4);
   MyInteger b(3);
   MyInteger c;

   c = a + b;
   c = c + 6;
   c = 8 + c;

   return 0;
}
```

Operator overloading

University of Exeter

Three considerations:

1. At least one of the operands of an overloaded operator must be of a user-defined type (usually this is a class).
2. Overloaded operators are just function calls, so cannot preserve
   - associativity e.g. $4 + 3 + 1 = (4 + 3) + 1 = 4 + (3 + 1)$
   - precedence rules e.g. $4 + 3 * 1 = 4 + (3 * 1) \neq (4 + 3) * 1$

   of the operator, according to its built-in semantics.
3. Standard && and || embody short-circuit semantics – the second operand is not evaluated if the truth of the expression is known. Cannot be preserved in operator overloading.

Operator overloading: []

University of Exeter

Example, without operator overloading:

```
class MyArray
{
    int data[100];
  public:
    int &elem(unsigned int i) { if (i > 99) error(); return data[i]; }
};

int main()
{
   MyArray a;
   a.elem(10) = 42;
   a.elem(12) += a.elem(13);

   return 0;
}
```

Operator overloading: []

Example, with operator overloading:

```
class MyArray
{
    int data[100];
  public:
    int &operator[] (unsigned int i) { if (i > 99) error(); return data[i]; }
};

int main()
{
   MyArray a;
   a[10] = 42;
   a[12] += a[13];

   return 0;
}
```

Operator overloading: [] vs ()

The difference between [] and () for subscripting:

- [] allows only one index
  - e.g. array[4]
- () allows multiple indices
  - e.g. matrix(5,1)

See here for more details and examples:
https://isocpp.org/wiki/faq/operator-overloading#matrix-subscript-op

Operator overloading: <<

Consider this date class again:

```
class MyDate
{
   int             year;
   unsigned int    month;
   unsigned int    day;
 public:
   MyDate(unsigned int,unsigned int,int);
}

MyDate::MyDate (unsigned int day, unsigned int month, int year)
{
   this->day   = day;
   this->month = month;
   this->year  = year;
}
```

```
int main ()
{
   // 20th March 2025
   MyDate dt(20,3,2025);
   return 0;
}
```

Operator overloading: <<

It would be very desirable to be able to do this:

```
MyDate dt(20,3,2025);
cout << "The date is: " << dt << ". Happy Birthday!" << endl;
```

to output a date in a particular format.

This requires us to overload the << operator

Operator overloading: <<

```
class MyDate
{
   int             year;
   unsigned int    month;
   unsigned int    day;
 public:
   MyDate(unsigned int,unsigned int,int);
   friend ostream &operator<<(ostream &os, MyDate &dt);
}

ostream &operator<<(ostream &os, MyDate &dt)
{
   os << dt.day << '/' << dt.month << '/' << dt.year;
   return os;
}
```

Operator overloading: <<

The overloaded << operator returns a reference to the original ostream object:

```
ostream &operator<<(ostream &os, MyDate &dt)
{
   os << dt.day << '/' << dt.month << '/' << dt.year;
   return os;
}
```

which means insertions can be combined:

```
cout << "The date is: " << dt << ". Happy Birthday!" << endl;
```

Wot, no operator overloading in Java?

University of Exeter

James Gosling said so:

I left out operator overloading as a **fairly personal choice** because I had seen too many people abuse it in C++.

James Gosling.
http://www.gotw.ca/publications/c_family_interview.htm

Many C++ design decisions have their roots in my dislike for forcing people to do things in some particular way [...] Often, I was tempted to outlaw a feature I personally disliked, I refrained from doing so because **I did not think I had the right to force my views on others**.

Bjarne Stroustrup.

The Design and Evolution of C++ (1.3 General Background)

Operator overloading

One other consideration:

Just because you can, doesn't mean you should!

See the ISO C++ website for some Rules of Thumb:
https://isocpp.org/wiki/faq/operator-overloading#law-of-least-surprise-op-ov

For example:

- If you define arithmetic operators, maintain the usual arithmetic identities. For example, if your class defines x + y and x - y, then x + y - y ought to return an object that is behaviourally equivalent to x.
- You should provide arithmetic operators only when they make logical sense to users.

Operator overloading

Some plausible examples:

- myString + yourString might concatenate two std::string objects
- myDate++ might increment a Date object by 1 day
- a[i] might access the *i*th element of a MyArray object
- m(i,j) might access element in row *i*, column *j* of a MyMatrix object
- matrix1 * matrix2 might perform matrix multiplication on two MyMatrix objects
- w + z, w – z, w*z , w/z might perform arithmetic on complex numbers w and z

Summary

- Friend functions
  - Enabling access to private and protected class members
- Operator overloading
  - Allows intuitive programming for the user
  - Most operators can be overloaded, but not all
  - Overloaded operators must involve a user class
  - Overloaded operators are just functions
  - Friend functions enable access to private class members by operator functions